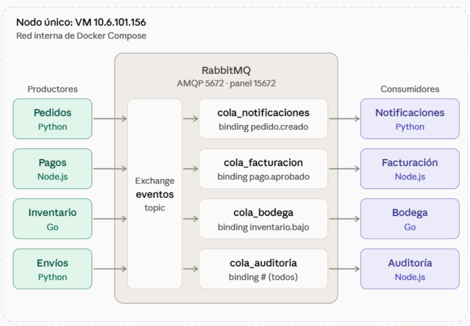
Taller Arquitectura Event-Driven Multilenguaje con RabbitMQ y Docker

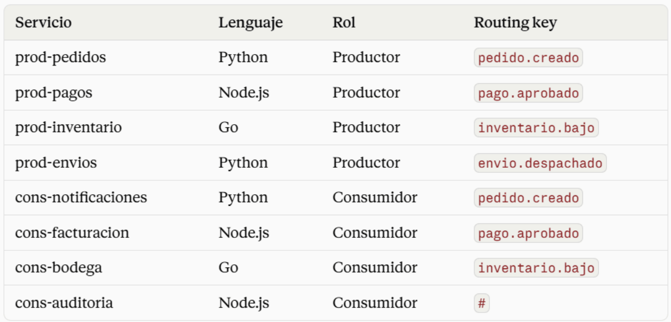
Integrantes:

- Angela Lozano
- Juan José Rangel

Escenario 1: centralizado (1 nodo)

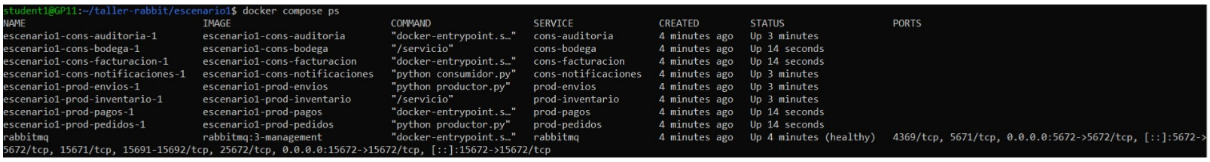
- Diagrama del escenario


Nodo único: VM 10.6.101.156
Red interna de Docker Compose
RabbitMQ
AMQP 5672 · panel 15672
Productores
Pedidos
Python
Pagos
Node.js
Inventario
Go
Envíos
Python
Exchange
eventos
topic
cola_notificaciones
binding pedido.creado
cola_facturacion
binding pago.aprobado
cola_bodega
binding inventario.bajo
cola_auditoria
binding # (todos)
Consumidores
Notificaciones
Python
Facturación
Node.js
Bodega
Go
Auditoría
Node.js



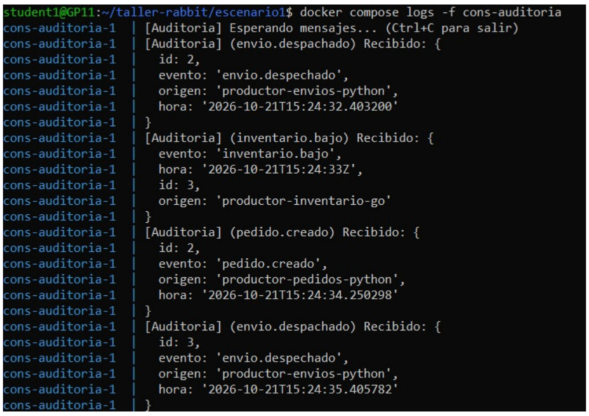
Servicio	Lenguaje	Rol	Routing key
prod-pedidos	Python	Productor	`pedido.creado`
prod-pagos	Node.js	Productor	`pago.aprobado`
prod-inventario	Go	Productor	`inventario.bajo`
prod-envios	Python	Productor	`envio.despachado`
cons-notificaciones	Python	Consumidor	`pedido.creado`
cons-facturacion	Node.js	Consumidor	`pago.aprobado`
cons-bodega	Go	Consumidor	`inventario.bajo`
cons-auditoria	Node.js	Consumidor	`#`

- Contenedores corriendo

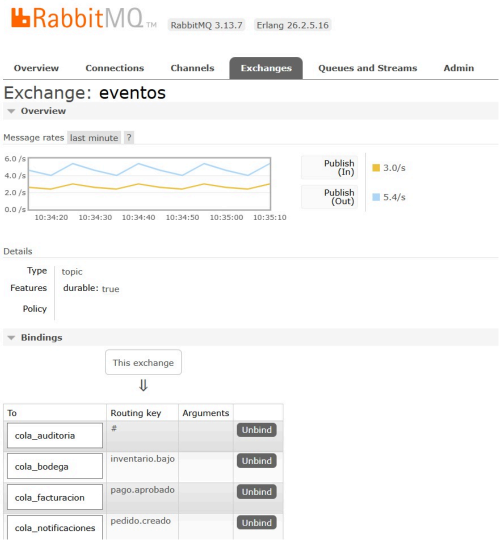


```
student1@GP11:~/taller-rabbit/escenario1$ docker compose ps
NAME                               IMAGE                            COMMAND                  SERVICE               CREATED         STATUS                   PORTS
escenario1-cons-auditoria-1        escenario1-cons-auditoria        "docker-entrypoint.s…"   cons-auditoria        4 minutes ago   Up 3 minutes
escenario1-cons-bodega-1           escenario1-cons-bodega           "/servicio"              cons-bodega           4 minutes ago   Up 14 seconds
escenario1-cons-facturacion-1      escenario1-cons-facturacion      "docker-entrypoint.s…"   cons-facturacion      4 minutes ago   Up 14 seconds
escenario1-cons-notificaciones-1   escenario1-cons-notificaciones   "python consumidor.py"   cons-notificaciones   4 minutes ago   Up 3 minutes
escenario1-prod-envios-1           escenario1-prod-envios           "python productor.py"    prod-envios           4 minutes ago   Up 3 minutes
escenario1-prod-inventario-1       escenario1-prod-inventario       "/servicio"              prod-inventario       4 minutes ago   Up 3 minutes
escenario1-prod-pagos-1            escenario1-prod-pagos            "docker-entrypoint.s…"   prod-pagos            4 minutes ago   Up 14 seconds
escenario1-prod-pedidos-1          escenario1-prod-pedidos          "python productor.py"    prod-pedidos          4 minutes ago   Up 14 seconds
rabbitmq                           rabbitmq:3-management            "docker-entrypoint.s…"   rabbitmq              4 minutes ago   Up 4 minutes (healthy)   4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, [::]:5672->
5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
```

Los 9 contenedores (4 productores, 4 consumidores y el broker RabbitMQ) corren en un único nodo, definidos en un solo docker-compose.yml. Los servicios se comunican por la red interna de Compose, usando el nombre del servicio rabbitmq como dirección. Se usa un exchange tipo topic llamado eventos; tres consumidores se enlazan a una routing key exacta y el de auditoría usa # para recibir todos los eventos.



```
student1@GP11:~/taller-rabbit/escenario1$ docker compose logs -f cons-auditoria
cons-auditoria-1  | [Auditoria] Esperando mensajes... (Ctrl+C para salir)
cons-auditoria-1  | [Auditoria] (envio.despachado) Recibido: {
cons-auditoria-1  |   id: 2,
cons-auditoria-1  |   evento: 'envio.despechado',
cons-auditoria-1  |   origen: 'productor-envios-python',
cons-auditoria-1  |   hora: '2026-10-21T15:24:32.403200'
cons-auditoria-1  | }
cons-auditoria-1  | [Auditoria] (inventario.bajo) Recibido: {
cons-auditoria-1  |   evento: 'inventario.bajo',
cons-auditoria-1  |   hora: '2026-10-21T15:24:33Z',
cons-auditoria-1  |   id: 3,
cons-auditoria-1  |   origen: 'productor-inventario-go'
cons-auditoria-1  | }
cons-auditoria-1  | [Auditoria] (pedido.creado) Recibido: {
cons-auditoria-1  |   id: 2,
cons-auditoria-1  |   evento: 'pedido.creado',
cons-auditoria-1  |   origen: 'productor-pedidos-python',
cons-auditoria-1  |   hora: '2026-10-21T15:24:34.250298'
cons-auditoria-1  | }
cons-auditoria-1  | [Auditoria] (envio.despachado) Recibido: {
cons-auditoria-1  |   id: 3,
cons-auditoria-1  |   evento: 'envio.despachado',
cons-auditoria-1  |   origen: 'productor-envios-python',
cons-auditoria-1  |   hora: '2026-10-21T15:24:35.405782'
cons-auditoria-1  | }
```


RabbitMQ TM
RabbitMQ 3.13.7
Erlang 26.2.5.16
Overview
Connections
Channels
Exchanges
Queues and Streams
Admin
Overview
Message rates
last minute
?
6.0 /s
4.0 /s
2.0 /s
0.0 /s
10:34:20
10:34:30
10:34:40
10:34:50
10:35:00
10:35:10
Publish (In)
3.0/s
Publish (Out)
5.4/s
Bindings
This exchange
⇓
Unbind

Exchange: eventos

Details

Type: topic
Features: durable: true
Policy:

To	Routing key	Arguments	
cola_auditoria	#		Unbind
cola_bodega	inventario.bajo		Unbind
cola_facturacion	pago.aprobado		Unbind
cola_notificaciones	pedido.creado		Unbind

Ventajas

- Es el más simple de desplegar: un solo docker-compose.yml y un solo comando levantan los 9 contenedores.
- Los servicios se comunican por la red interna de Docker usando el nombre del servicio (rabbitmq), sin depender de direcciones IP.
- No hay latencia de red física entre productores, consumidores y broker.
- Es fácil de depurar: todos los logs y contenedores están en una sola máquina.

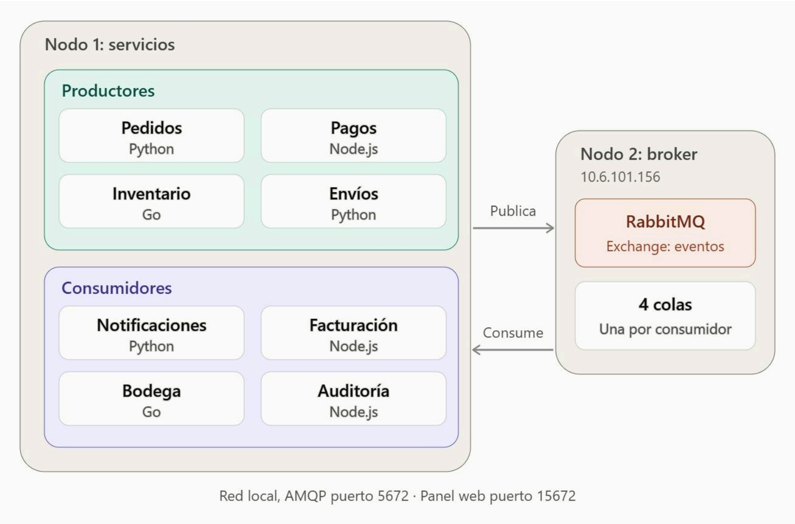
Desventajas

- Tiene un punto único de fallo: si la VM cae, cae todo el sistema.
- Productores, consumidores y broker compiten por la misma CPU, memoria y disco.
- No permite escalar horizontalmente ni demuestra el desacoplamiento físico propio de una arquitectura event-driven.

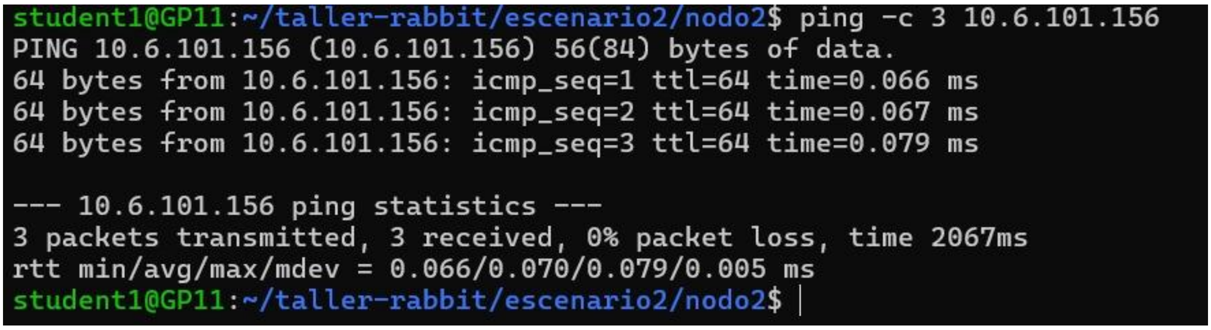
Escenario 2: distribuido en 2 nodos

Nodo 1: productores y consumidores. Nodo 2: broker RabbitMQ.

- Diagrama del escenario

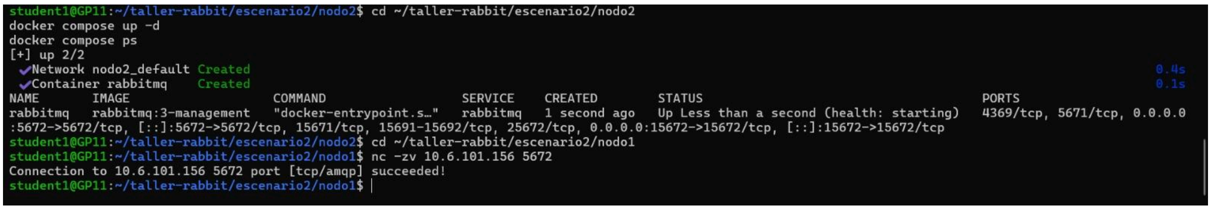

Nodo 1: servicios
Productores
Pedidos
Python
Pagos
Node.js
Inventario
Go
Envíos
Python
Consumidores
Notificaciones
Python
Facturación
Node.js
Bodega
Go
Auditoría
Node.js
Publica
Consume
Nodo 2: broker
10.6.101.156
RabbitMQ
Exchange: eventos
4 colas
Una por consumidor
Red local, AMQP puerto 5672 · Panel web puerto 15672

- Conexión entre los nodos con ping



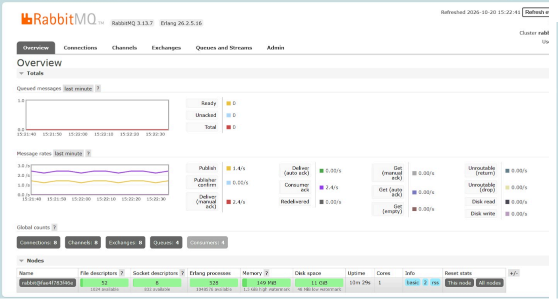
```
student1@GP11:~/taller-rabbit/escenario2/nodo2$ ping -c 3 10.6.101.156
PING 10.6.101.156 (10.6.101.156) 56(84) bytes of data.
64 bytes from 10.6.101.156: icmp_seq=1 ttl=64 time=0.066 ms
64 bytes from 10.6.101.156: icmp_seq=2 ttl=64 time=0.067 ms
64 bytes from 10.6.101.156: icmp_seq=3 ttl=64 time=0.079 ms

--- 10.6.101.156 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2067ms
rtt min/avg/max/mdev = 0.066/0.070/0.079/0.005 ms
student1@GP11:~/taller-rabbit/escenario2/nodo2$ 
```

- El nodo 2 funcionando con el broker y el nodo 1 conectado exitosamente



```
student1@GP11:~/taller-rabbit/escenario2/nodo2$ cd ~/taller-rabbit/escenario2/nodo2
docker compose up -d
docker compose ps
[+] up 2/2
 ✔Network nodo2_default Created                                                                                    0.4s
 ✔Container rabbitmq    Created                                                                                    0.1s
NAME       IMAGE                 COMMAND                   SERVICE    CREATED         STATUS                                   PORTS
rabbitmq   rabbitmq:3-management   "docker-entrypoint.s…"   rabbitmq   1 second ago    Up Less than a second (health: starting)   4369/tcp, 5671/tcp, 0.0.0.0
:5672->5672/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
student1@GP11:~/taller-rabbit/escenario2/nodo2$ cd ~/taller-rabbit/escenario2/nodo1
student1@GP11:~/taller-rabbit/escenario2/nodo1$ nc -zv 10.6.101.156 5672
Connection to 10.6.101.156 5672 port [tcp/amqp] succeeded!
student1@GP11:~/taller-rabbit/escenario2/nodo1$ |
```

- Conexiones en Rabbit MQ

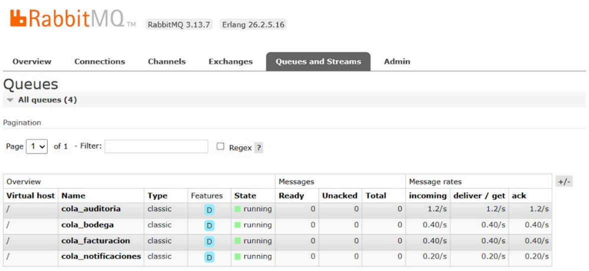

RabbitMQ
RabbitMQ 3.13.7
Erlang 26.2.5.16
Refreshed 2026-10-20 15:22:41
Refresh e
Cluster rabl
Us
Overview
Connections
Channels
Exchanges
Queues and Streams
Admin
Overview
Totals
Queued messages
last minute
?
1.0
0.0
15:21:40
15:21:50
15:22:00
15:22:10
15:22:20
15:22:30
Message rates
3.0 /s
2.0 /s
1.0 /s
0.0 /s
Global counts
Nodes
+/-

Ready	Unacked	Total
0	0	0

Publish	Publisher confirm	Deliver (manual ack)	Deliver (auto ack)	Consumer ack	Redelivered	Get (manual ack)	Get (auto ack)	Get (empty)	Unroutable (return)	Unroutable (drop)	Disk read	Disk write
1.4/s	0.00/s	2.4/s	0.00/s	2.4/s	0.00/s	0.00/s	0.00/s	0.00/s	0.00/s	0.00/s	0.00/s	0.00/s

Connections: 8 Channels: 8 Exchanges: 8 Queues: 4 Consumers: 4

Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Cores	Info	Reset stats
rabbit@fae4f783f46e	52 1024 available	8 832 available	528 1048576 available	149 MiB 1.5 GiB high watermark	11 GiB 48 MiB low watermark	10m 29s	1	basic 2 rss	This node All nodes


RabbitMQ
RabbitMQ 3.13.7
Erlang 26.2.5.16
Overview
Connections
Channels
Exchanges
Queues and Streams
Admin
Queues
All queues (4)
Pagination
Page 1 of 1 - Filter:
☐ Regex ?
+/-

Overview					Messages			Message rates		
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack
/	cola_auditoria	classic	D	running	0	0	0	1.2/s	1.2/s	1.2/s
/	cola_bodega	classic	D	running	0	0	0	0.40/s	0.40/s	0.40/s
/	cola_facturacion	classic	D	running	0	0	0	0.40/s	0.40/s	0.40/s
/	cola_notificaciones	classic	D	running	0	0	0	0.20/s	0.20/s	0.20/s

Ventajas

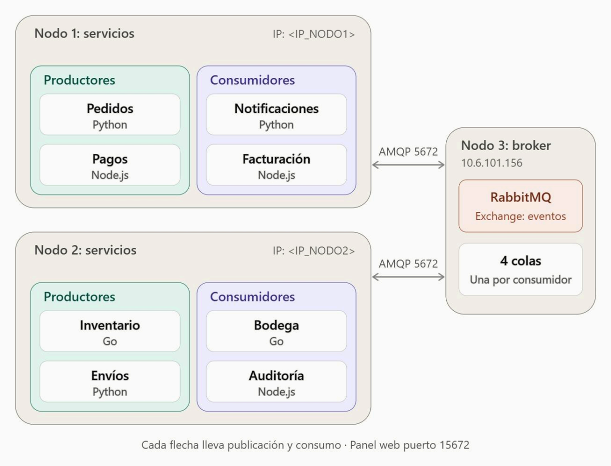
- El broker queda aislado en su propio nodo, con recursos dedicados, así que una carga alta en los servicios no lo afecta directamente.
- Una falla de los servicios no tumba el broker, y los mensajes ya almacenados en las colas se conservan.
- Se acerca más a un entorno real, donde el broker suele ser una pieza de infraestructura independiente.

Desventajas

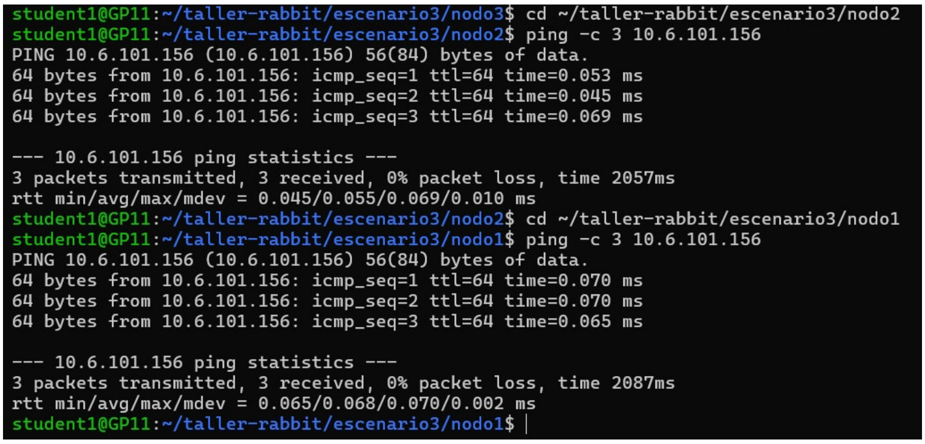
- El broker sigue siendo un punto único de fallo: si el Nodo 2 cae, ningún servicio puede comunicarse.
- Aparece la latencia de red entre los servicios y el broker.
- Hay que configurar el acceso por red: usar la IP del broker en `RABBIT_HOST`, abrir los puertos 5672 y 15672 y revisar el firewall.
- Se gestionan 2 archivos `docker-compose.yml` y el despliegue se coordina entre dos máquinas.
- Si los servicios arrancan antes que el broker, fallan al conectarse. Ya no sirve `depends_on`, porque el broker no está en el mismo compose.

Escenario 3: distribuido en 3 nodos

- Diagrama del escenario

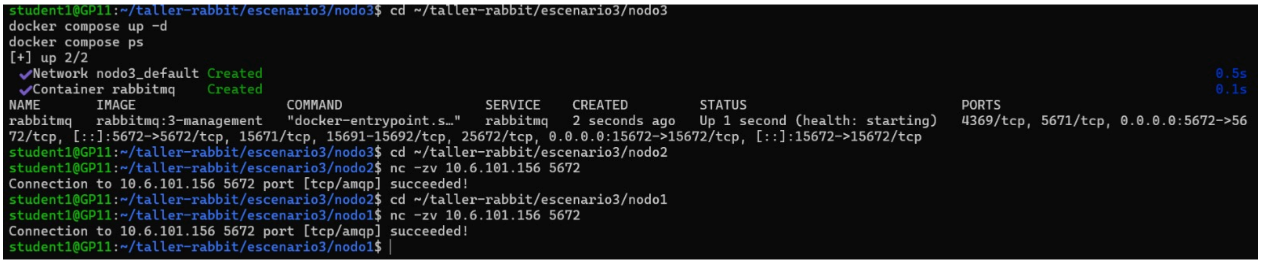

Nodo 1: servicios
IP: <IP_NODO1>
Productores
Pedidos
Python
Pagos
Node.js
Consumidores
Notificaciones
Python
Facturación
Node.js
AMQP 5672
Nodo 3: broker
10.6.101.156
RabbitMQ
Exchange: eventos
4 colas
Una por consumidor
Nodo 2: servicios
IP: <IP_NODO2>
Productores
Inventario
Go
Envíos
Python
Consumidores
Bodega
Go
Auditoría
Node.js
AMQP 5672
Cada flecha lleva publicación y consumo · Panel web puerto 15672

- Pings de los nodos



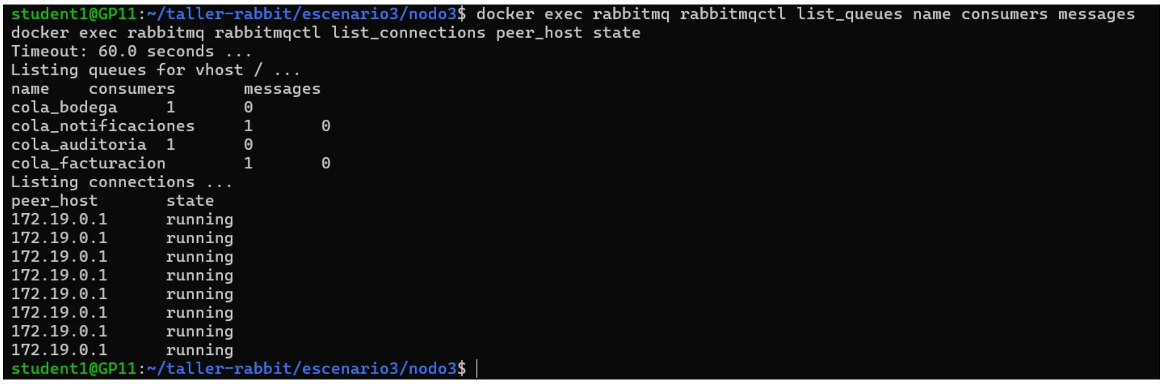
```
student1@GP11:~/taller-rabbit/escenario3/nodo3$ cd ~/taller-rabbit/escenario3/nodo2
student1@GP11:~/taller-rabbit/escenario3/nodo2$ ping -c 3 10.6.101.156
PING 10.6.101.156 (10.6.101.156) 56(84) bytes of data.
64 bytes from 10.6.101.156: icmp_seq=1 ttl=64 time=0.053 ms
64 bytes from 10.6.101.156: icmp_seq=2 ttl=64 time=0.045 ms
64 bytes from 10.6.101.156: icmp_seq=3 ttl=64 time=0.069 ms

--- 10.6.101.156 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2057ms
rtt min/avg/max/mdev = 0.045/0.055/0.069/0.010 ms
student1@GP11:~/taller-rabbit/escenario3/nodo2$ cd ~/taller-rabbit/escenario3/nodo1
student1@GP11:~/taller-rabbit/escenario3/nodo1$ ping -c 3 10.6.101.156
PING 10.6.101.156 (10.6.101.156) 56(84) bytes of data.
64 bytes from 10.6.101.156: icmp_seq=1 ttl=64 time=0.070 ms
64 bytes from 10.6.101.156: icmp_seq=2 ttl=64 time=0.070 ms
64 bytes from 10.6.101.156: icmp_seq=3 ttl=64 time=0.065 ms

--- 10.6.101.156 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2087ms
rtt min/avg/max/mdev = 0.065/0.068/0.070/0.002 ms
student1@GP11:~/taller-rabbit/escenario3/nodo1$ |
```

- Nodos 1 y 2 funcionando y conectados exitosamente



```
student1@GP11:~/taller-rabbit/escenario3/nodo3$ cd ~/taller-rabbit/escenario3/nodo3
docker compose up -d
docker compose ps
[+] up 2/2
 ✔Network nodo3_default Created                                                                                                     0.5s
 ✔Container rabbitmq    Created                                                                                                     0.1s
NAME       IMAGE                   COMMAND                  SERVICE    CREATED          STATUS                            PORTS
rabbitmq   rabbitmq:3-management   "docker-entrypoint.s…"   rabbitmq   2 seconds ago    Up 1 second (health: starting)    4369/tcp, 5671/tcp, 0.0.0.0:5672->56
72/tcp, [::]:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
student1@GP11:~/taller-rabbit/escenario3/nodo3$ cd ~/taller-rabbit/escenario3/nodo2
student1@GP11:~/taller-rabbit/escenario3/nodo2$ nc -zv 10.6.101.156 5672
Connection to 10.6.101.156 5672 port [tcp/amqp] succeeded!
student1@GP11:~/taller-rabbit/escenario3/nodo2$ cd ~/taller-rabbit/escenario3/nodo1
student1@GP11:~/taller-rabbit/escenario3/nodo1$ nc -zv 10.6.101.156 5672
Connection to 10.6.101.156 5672 port [tcp/amqp] succeeded!
student1@GP11:~/taller-rabbit/escenario3/nodo1$ |
```

- Información de rabbitMQ desde consola



```
student1@GP11:~/taller-rabbit/escenario3/nodo3$ docker exec rabbitmq rabbitmqctl list_queues name consumers messages
docker exec rabbitmq rabbitmqctl list_connections peer_host state
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    consumers       messages
cola_bodega     1       0
cola_notificaciones     1       0
cola_auditoria  1       0
cola_facturacion        1       0
Listing connections ...
peer_host       state
172.19.0.1      running
172.19.0.1      running
172.19.0.1      running
172.19.0.1      running
172.19.0.1      running
172.19.0.1      running
172.19.0.1      running
172.19.0.1      running
student1@GP11:~/taller-rabbit/escenario3/nodo3$ |
```

Ventajas

- La carga de los servicios se reparte entre dos nodos, y el broker mantiene recursos exclusivos.
- Es el escenario que mejor demuestra el desacoplamiento: servicios en máquinas distintas se comunican sin conocerse, sólo a través del exchange.
- La falla de un nodo de servicios no detiene al otro, así que hay más tolerancia a fallos parciales.
- Es más escalable: se pueden sumar más nodos de productores o consumidores sin tocar el broker.

Desventajas

- Es el de mayor complejidad operativa: 3 archivos `docker-compose.yml`, 3 máquinas que sincronizar y más puntos donde algo puede fallar.
- Más tráfico por la red y más sensibilidad a su calidad.
- Seguimos con un solo broker, así que sigue existiendo un punto único de fallo. Resolverlo requeriría un clúster de RabbitMQ.
- Depurar es más difícil, porque los logs están repartidos en tres máquinas.